

Call Centre Workforce Planning Using Erlang C: The Deterministic Approach vs. the Monte Carlo Approach

Zakaria Daoud

April 21, 2025

1 What is Erlang C?

ErlangC is a fundamental model used to calculate call center workforce planning. Developed by A.K. Erlang in the early 20th century, the model estimates the probability that an arriving call will need to wait for service in an environment with multiple agents. Widely employed to determine the minimum staffing levels required to achieve a target service level, we can improve it by simulate a real-world call center using Monte Carlo methods.

At its core, Erlang C models an $M/M/c$ queueing system, characterized by the following assumptions:

- **Poisson arrivals:** Calls arrive randomly and independently, following a Poisson process.
- **Exponential service times:** The duration of each call follows an exponential distribution, implying a memoryless service process.
- **Multiple homogeneous servers:** There are c identical servers (agents), each capable of handling one call at a time.
- **Infinite queue capacity and no abandonment:** Calls that cannot be immediately served are queued indefinitely until an agent is available.
- **First-Come-First-Served (FCFS):** Calls are served in the order they arrive, with no prioritization.

While these assumptions offer a mathematically tractable framework, they also introduce certain limitations:

- *Arrival variability:* Real-world call centres often experience time-dependent call volumes (e.g., peak hours), which deviate from the constant-rate Poisson assumption.
- *Non-exponential service durations:* Call durations are frequently better described by log-normal or gamma distributions, particularly in centres with diverse call types.
- *Caller patience:* The assumption of zero abandonment is often unrealistic; customers may hang up before reaching an agent, especially during long wait times.

- *Agent heterogeneity*: Variations in agent performance or specialization are not accounted for, yet they can influence overall efficiency.
- *Steady-state conditions*: The model assumes long-run equilibrium, which may not capture transient effects such as shift changes or sudden call spikes.

Despite these simplifications, the Erlang C model remains a cornerstone of operational planning due to its ability to approximate staffing requirements with reasonable accuracy under average conditions. When paired with simulation techniques, it can be extended to model performance variability, customer abandonment, and other complexities of modern call centre environments. Its application is thus critical for designing efficient operations and ensuring service-level compliance.

2 Mathematical Framework

The Erlang C model is rooted in the $M/M/c$ queueing system, a classical framework in operations research. In this system, arrivals follow a Poisson process, service times are exponentially distributed, and there are c identical servers (agents). The model assumes an infinite queue and no call abandonment, focusing on steady-state performance.

The central quantity of interest is the **probability that an arriving call must wait**, denoted $P(\text{wait})$. This occurs when all c servers are busy upon a call's arrival. The formula is given by:

$$P(\text{wait}) = \frac{\frac{A^c}{c!} \cdot \frac{c}{c-A}}{\sum_{n=0}^{c-1} \frac{A^n}{n!} + \frac{A^c}{c!} \cdot \frac{c}{c-A}},$$

where:

- λ is the average arrival rate (calls per unit time),
- t is the average handling time per call (in the same time units as λ),
- $A = \lambda \cdot t$ is the *offered load* in Erlang, representing the expected number of busy agents,
- c is the number of agents (servers) available.

The numerator $\frac{A^c}{c!} \cdot \frac{c}{c-A}$ captures the steady-state probability that all servers are occupied and an additional call arrives (i.e., the system is at capacity), while the denominator normalizes over all system states. The expression becomes numerically unstable for large c due to the factorial, which motivates the use of logarithmic computation techniques in high-volume environments.

In workforce planning, another critical metric is the **service level**, defined as the probability that a call is answered within a specified threshold time T . It is computed as:

$$SL(T) = 1 - P(\text{wait}) \cdot \exp[-(c - A) \mu T],$$

where:

- T is the target answer time (in minutes or seconds),

- $\mu = 1/t$ is the service rate (calls per unit time per agent),
- $(c - A)\mu$ is the effective rate at which queued calls are served once they wait.

This expression assumes that the time a call spends in queue follows an exponential distribution, conditional on having to wait. It links service goals (e.g., “80% of calls answered within 20 seconds”) directly to staffing levels and load.

Additionally, we introduce two related performance indicators:

- **Occupancy:** $\rho = \frac{A}{c}$, the fraction of time agents are busy. High occupancy improves efficiency but may lead to longer queues and burnout.
- **Average wait time (optional):**

$$W_q = \frac{P(\text{wait})}{c\mu - \lambda},$$

which estimates the mean time a customer waits in queue before being served. This expression is valid under the assumption that $c\mu > \lambda$.

Together, these formulas form the analytical backbone of Erlang C-based planning. They allow for direct translation of business objectives—such as maintaining a target service level or occupancy—into operational staffing decisions. In the next section, we enhance this deterministic model with a Monte Carlo simulation to better account for real-world variability.

3 Simulation

3.1 The Deterministic Approach

The code *ErlangC_calculator.py* begin with the definition of the needed functions to simulate the ErlangC model. The functions are listed as follow:

1. *intensity(rate, duration, interval=60)*: this function compute the offered intensity in a deterministic point-of-view, where the rate is the number of calls in a defined time interval (here 60 minutes for 1 hour), and the duration is the mean call duration in seconds.
2. *erlang_c(agents, rate, duration, interval=60)*: used to calculate the deterministic ErlangC formula. This function will return the ErlangC probability that a call must wait.
3. *service_level(agents, rate, duration, target, interval=60)*: to calculate the probability that a call is answered within 'target' seconds. This function will return the service level in percentage, ranging from 0 to 1.
4. *resource(rate, duration, target, gos_target, interval=60)*: to determine the minimal number of agents needed to achieve a desired service level. Where gos_target is the desired service level, this function will return a tuple of agents and service level meeting or exceeding gos_target.

5. *calculate_shrinkage(shift_length_min, breaks_min, meetings_min, training_min)*: this function calculate the shrinkage if the user wants to, if not the user can put the shrinkage value as an input without calculating it (we advise calculating the shrinkage if the user wants to continue with the Monte Carlo simulation).

3.2 The Monte Carlo Approach

In addition to the deterministic model, a Monte Carlo simulation is incorporated to capture variability in real-world call centre operations. In our simulation:

- **Intensity:** The offered load is simulated by drawing random samples for both the call rate and call duration. In our implementation, the intensity is computed as:

$$\text{intensity} = \left(\frac{N(\text{rate_m}, \text{rate_sd})}{60 \times \text{interval}} \right) \times N(\text{duration_m}, \text{duration_sd}),$$

where $N(\cdot)$ denotes a normally distributed random variable.

- **Erlang C Simulation:** For each simulation run, the Erlang C probability is calculated iteratively based on the simulated intensity.
- **Service Level:** A simulated service level is then computed using the estimated $P(\text{wait})$, incorporating additional random variation to model the target answer time and call duration variability.

By running the simulation (e.g., 1000 iterations), we obtain a distribution of service level outcomes, which provides insight into the uncertainty and variability in performance.

4 Results

The results are obtained using both deterministic and Monte Carlo approaches:

- **Deterministic Results:** The deterministic model calculates a single staffing level required to meet a target service level. It further computes effective agents (after accounting for shrinkage), occupancy (the proportion of time agents are busy), and an approximate call abandonment rate.
- **Monte Carlo Simulation Results:** The simulation produces a distribution of service levels across 1000 runs. Summary statistics such as the minimum, maximum, mean, and median service level are computed and visualized using a histogram.

These results allow decision makers to understand not only the average expected performance but also the variability and risk associated with call centre operations.

5 Differences Between My Code and the Provided Erlang C Calculator

Several enhancements distinguish our implementation from the standard Erlang C calculator:

- a) **Integration of Simulation:** While traditional calculators provide a deterministic output based on average values, our approach integrates a Monte Carlo simulation that captures the inherent variability in call arrivals and service times.
- b) **Extended Parameters:** Our code incorporates additional real-world factors such as shrinkage (reflecting agent unavailability), occupancy limits, and caller patience (affecting call abandonment). These enhancements yield a more realistic workforce planning tool.
- c) **Numerical Stability:** To handle high traffic volumes, we implement logarithmic computations (using a log-sum-exp technique) in our deterministic model, thereby mitigating numerical overflow issues.
- d) **Enhanced Visualization:** Our merged code outputs both numerical summaries and graphical representations (line plots and histograms), providing a comprehensive view of performance metrics and aiding in decision-making.

6 Conclusion

Both approaches have pros and cons; however, we need to make two major precisions:

- Both models are correct; it depends on which one the user will prefer. For example, if the call center needs less than 20 agents, then the deterministic approach will produce almost the same results as the Monte Carlo Approach. But if there are more than 60 agents, the Monte Carlo approach is preferable in order to calculate the right number of agents needed for the call center workforce.
-